Extending prototypes is a powerful technique that allows you to change the behavior of any object in CoffeeScript. You have the power. Be careful how you use it.

5.6.3. Don’t modify objects you don’t own

By extending the built-in `Date` object, you’re changing it for the entire program. If you’re writing a library that other people are going to use, then you’re changing things for other people’s entire programs. You don’t know what those other people are doing in their programs, so extending built-ins has a high potential for breaking other people’s code. Modifying built-in prototypes is a bit like breaking into somebody’s house and leaving stuff there. This works only if you’re Santa Claus.

One alternative to modifying a built-in prototype is to write your own class that `extends` the built-in and use that instead:

```
class ExtendedDate extends Date
  daysFromToday: ->
    millisecondsInDay = 86400000
    today = new Date
    diff = @ - today
    Math.floor diff/millisecondsInDay
```

You should trade the convenience of extending a built-in for its danger on a case-by-case basis.

You’ve seen how to declare a class that allows you to create objects from the class. You’ve also seen how classes are just functions and prototypes when you lift the veil and look underneath. Class convenience comes with some rigidity, meaning you go back to raw prototypes when you need to dynamically extend things. This is an important lesson worth repeating.

The class sweet spot

If you’re mostly creating certain kinds of objects with some extensions, then use classes. They work very well for that. If, however, your problem